Smart Outing Planner

Final Website Project Report

Course: CCSW321 - Web Development

Project Type: Full Web Application

Submission: Final Combined Submission

Academic Year: 2025-2026

Backend: Node.js / Express

Database: Firebase Firestore

Student Name	Student ID	Main Contribution
Amir Al-Ghamdi	2340533	Back-end & Firebase
Abdulmalik Al-Zahrani	2340534	Front-end & Testing
Faisal Al-Orabi	2341120	Front-end & Testing / Technical Lead

1. Project Overview

Smart Outing Planner is a complete web application that helps users create suitable outing plans based on group type, number of guests, preferred atmosphere, time, and budget. The system provides a Saudi-inspired visual identity and a practical workflow that allows users to generate plans, refine activities, save plans, and revisit them later.

The website includes multiple HTML pages, separated CSS and JavaScript files, a Node/Express backend, Firebase Authentication, Firestore database integration, frontend and backend validation, and Arabic/English language support.

Main Objectives

- Develop a unique and interactive website idea rather than a simple static website.
- Use HTML5, CSS3, JavaScript, backend APIs, and a database properly.
- Collect and validate user input on both frontend and backend layers.
- Save and retrieve data using Firestore collections.
- Provide a responsive interface that works on small, medium, and large screen sizes.

2. Technology Stack

Layer	Technology / Purpose
Frontend Structure	HTML5 semantic pages using header, nav, main, section, table, form, and footer.
Styling	CSS3 separated in css/styles.css with responsive layouts and a Saudi heritage theme.
Frontend Logic	JavaScript modules for planner, results, validation, saved plans, plan details, Firebase, and i18n.
Backend	Node.js and Express routes, controllers, and validation modules.
Authentication	Firebase Authentication for login and account creation.
Database	Firebase Firestore collections: activities, savedPlans, contactMessages, and users.
Internationalization	Arabic/English support through js/i18n.js.

3. Project Structure

The project uses a clear folder structure that separates HTML, CSS, media, JavaScript, and backend code.

```
web_dev/
  backend/      Node/Express backend routes, controllers, validation, Firebase Admin setup
  css/          Main stylesheet and responsive styling
  firebase/     Firebase clarification notes
  html/         Website HTML pages
  js/           Frontend JavaScript modules
  media/        Images and visual assets
  README.md     Setup and project documentation
```

4. Requirement Compliance Summary

Requirement Area	Implementation	Status
Folder structure	HTML, CSS, JS, media, and backend	Completed

	folders are separated.	
Pages	Website includes index, about-us, contact-us, planner, results, details, saved plans, and authentication pages.	Completed
Semantic HTML5	Pages use semantic structures and proper form/table elements.	Completed
Separated CSS	Styling is centralized in css/styles.css.	Completed
Separated JavaScript	Frontend behavior is implemented in separate JS files.	Completed
Frontend validation	HTML attributes and js/validation.js validate contact inputs and display user messages.	Completed
Backend validation	Backend validation checks and cleans input before database operations.	Completed
Database integration	Firestore is used to save and display users, contact messages, activities, and saved plans.	Completed
Responsiveness	The website was tested at 576px, 768px, and 992px.	Completed
HTML validation	Nu HTML Checker results show no errors or warnings.	Completed

5. Website Pages and Screenshots



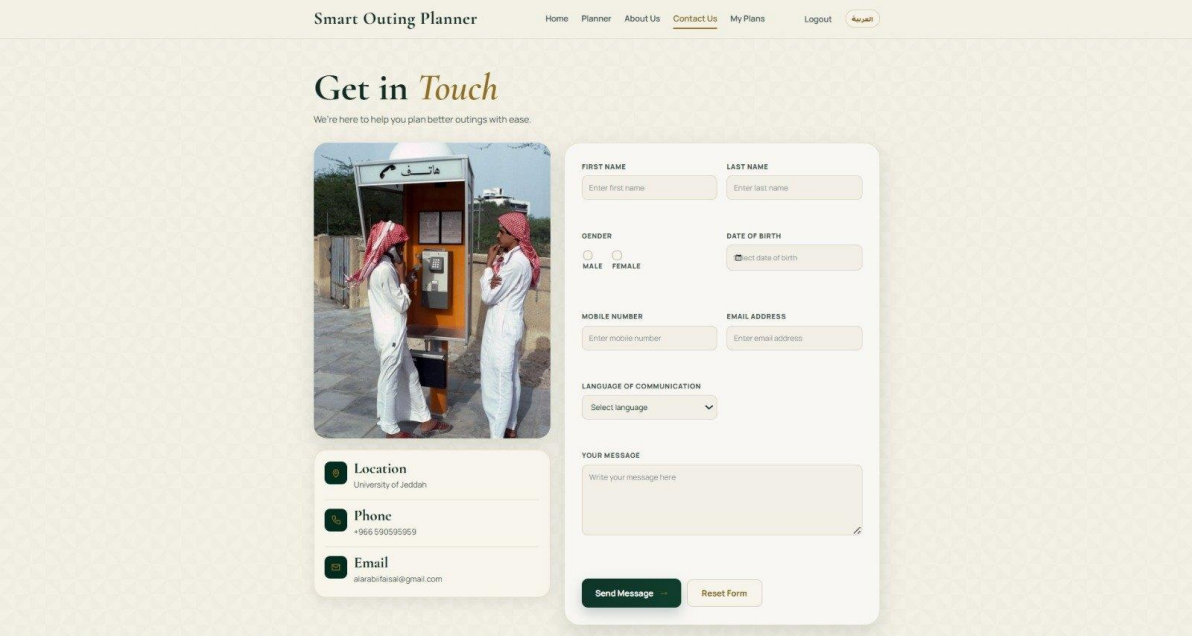
Home Page - landing section



Home Page - project value section



About Us - team members section



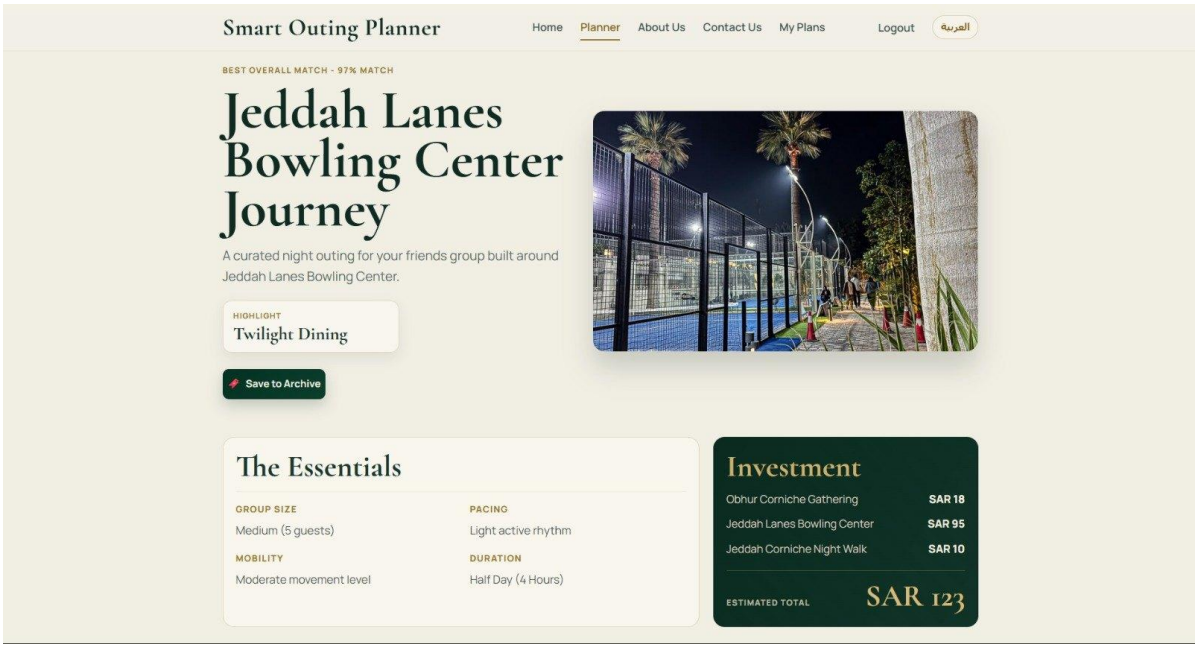
Contact Us - contact form page



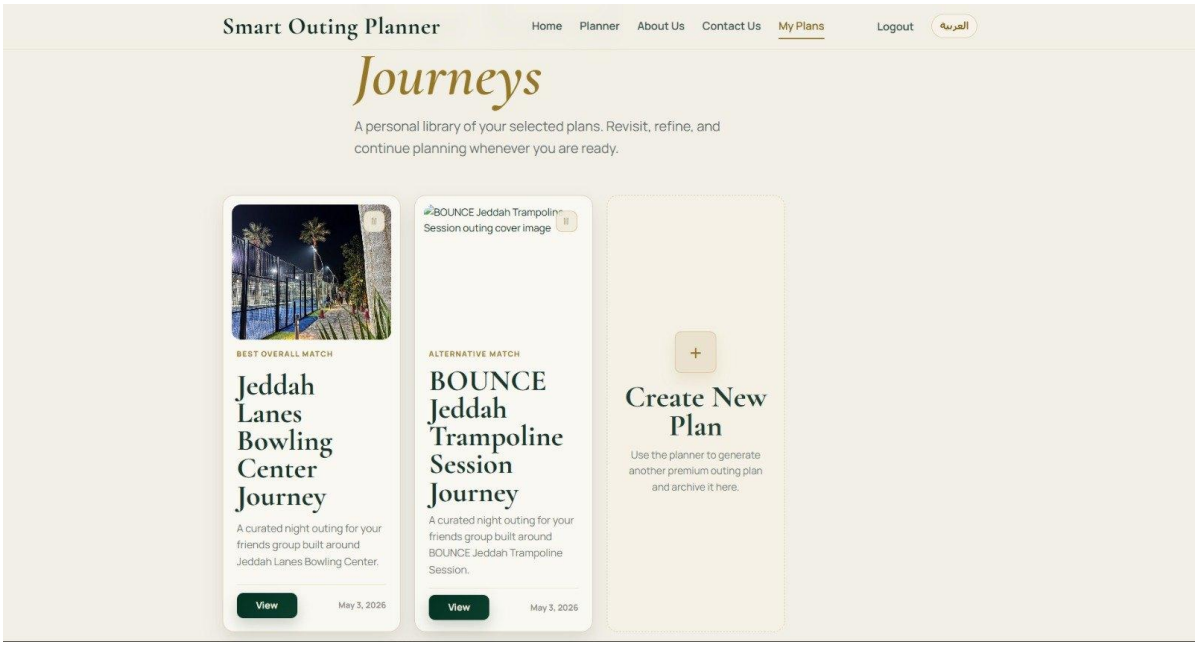
Planner Page - user preference selection



Results Page - generated outing plan



Plan Details Page - selected plan details



Saved Plans Page - user saved plans archive



Authentication Page - login screen

6. Main Dynamic Features

The website is not limited to static content. It supports a complete planning workflow with generated results, refinement, saving, and database retrieval.

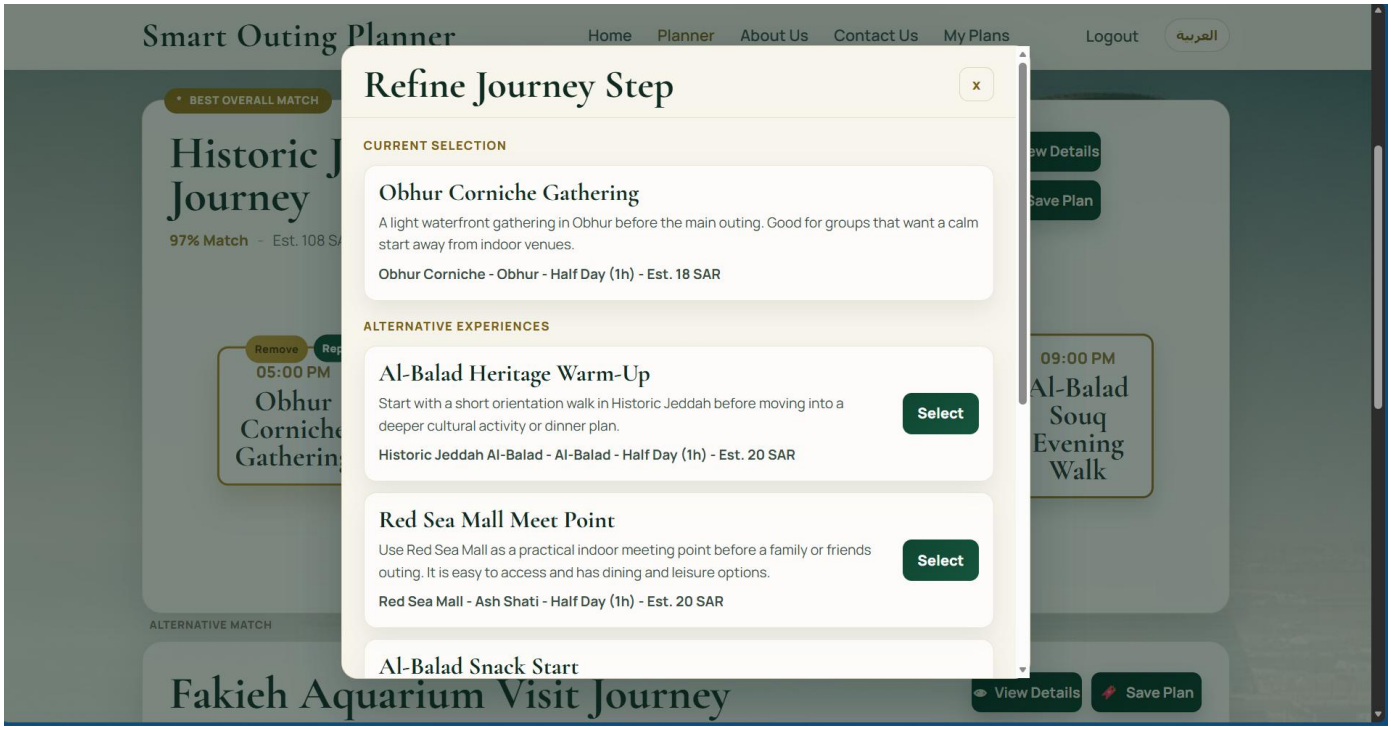
6.1 Plan Generation Flow

1. The user selects preferences in the planner page.
2. Frontend JavaScript collects the selected values.
3. The request is sent to the backend through the /api/plans/generate endpoint.
4. The backend validates the input and reads active activities from Firestore.

5. The planning logic creates suitable primary and alternative plans.
6. The results page displays the generated route with cost, duration, and match percentage.

6.2 Replace and Remove Activities

The results page allows users to refine a route. Users can replace one activity with a suitable alternative or remove an activity from the route. After each action, the page recalculates cost, duration, route steps, and saved state.



The replace modal evidence was captured during testing, and the final application successfully displayed alternative experiences and updated the current route.

6.3 Save and View Saved Plans

Saving a plan is protected by authentication. The frontend sends the authenticated user token to the backend. The backend verifies the user and saves the plan in Firestore with the user ID. The Saved Plans page later retrieves only plans belonging to the logged-in user.

7. User Input Validation

The project implements validation on both frontend and backend. Frontend validation improves user experience, while backend validation protects the database from invalid or manipulated input.

7.1 Contact Form Validation

- First name and last name are required and must match the allowed pattern.
- Gender is required.
- Mobile number must match Saudi mobile format.
- Date of birth must not be in the future.
- Email must match email format.
- Language of communication is required.
- Message is required and must meet length limits.

7.2 Frontend Validation Code Snippet

```
const namePattern = /^[A-Za-z\u0600-\u06FF\s'-]{2,30}$/;
const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
const saudiMobilePattern = /^(?:\+9665[0-9]{8}|05[0-9]{8})$/;

function validateContactForm(form) {
  const cleanMessage = buildCleanContactMessage({
    firstName: getFieldValue(form, "firstName"),
    lastName: getFieldValue(form, "lastName"),
    gender: getFieldValue(form, "gender"),
    mobile: getFieldValue(form, "mobile"),
    birthDate: getFieldValue(form, "birthDate"),
    email: getFieldValue(form, "email"),
    language: getFieldValue(form, "language"),
    message: getFieldValue(form, "message"),
  });

  const errors = getContactValidationErrors(cleanMessage);
  clearContactErrors();
  Object.keys(errors).forEach((fieldName) => {
    setFieldError(fieldName, errors[fieldName]);
  });

  return {
    isValid: Object.keys(errors).length === 0,
    data: cleanMessage,
  };
}
```

7.3 Backend Validation Code Snippet

```
function validateContactMessage(messageData) {
  const cleanMessage = buildCleanContactMessage(messageData);
  const errors = getContactValidationErrors(cleanMessage);
  const firstError = Object.values(errors)[0];

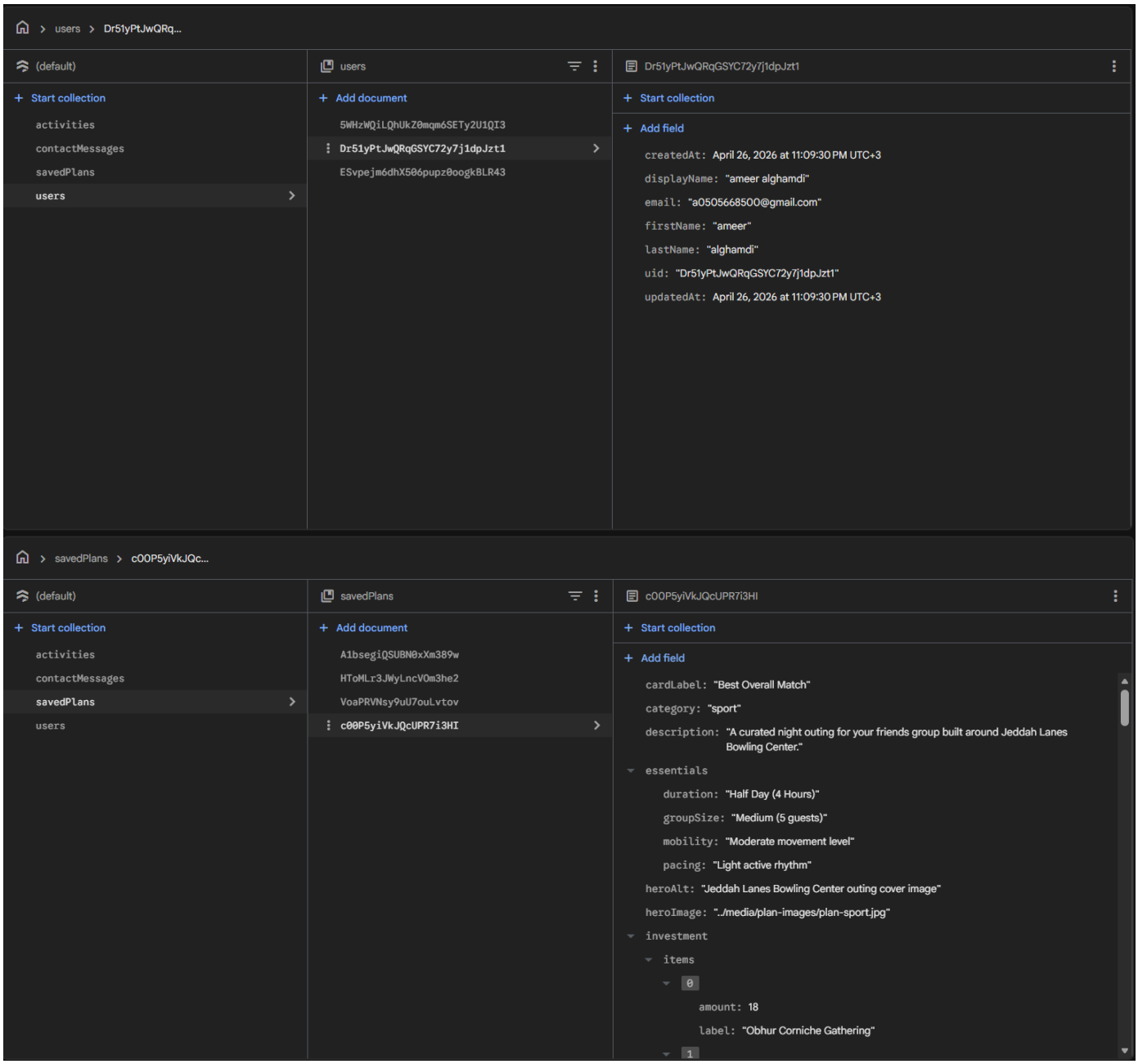
  if (firstError) {
    return { success: false, error: firstError };
  }

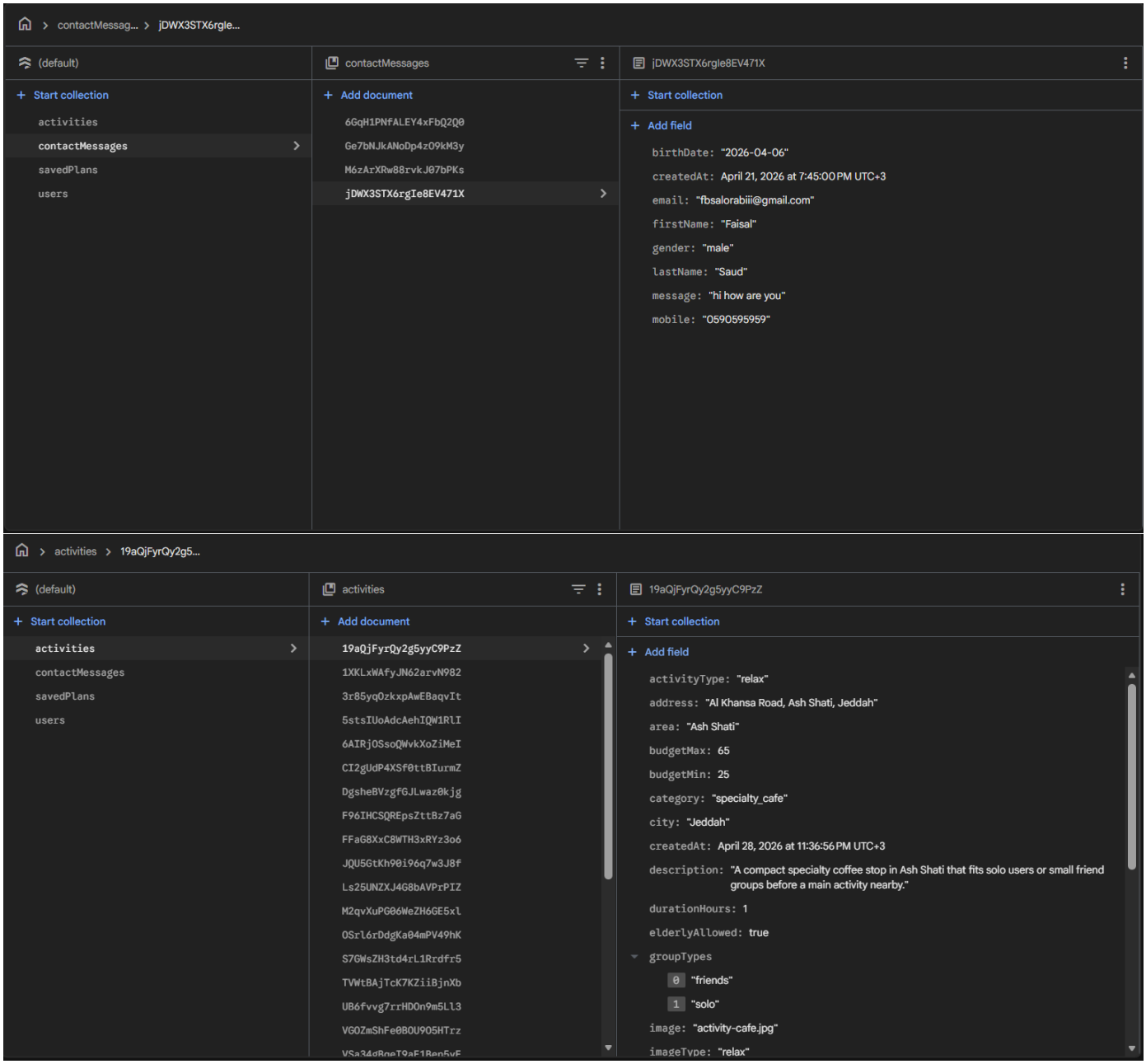
  return { success: true, data: cleanMessage };
}
```

8. Database Integration

The project uses Firebase Firestore as the database. The backend uses Firebase Admin SDK to perform database operations securely and consistently.

Collection	Purpose
activities	Stores active outing activities used by the planning engine.
savedPlans	Stores generated plans saved by authenticated users.
contactMessages	Stores contact form submissions.
users	Stores user profile data linked to Firebase Authentication UID.


users > Dr51yPtJwQRq...
(default)
Start collection
activities
contactMessages
savedPlans
users
users
Add document
5WHzWQiLQhUkZ0mqm6SETy2U1QI3
Dr51yPtJwQRqGSYC72y7j1dpJzt1
ESvpejm6dhX506pupz0oogkBLR43
Dr51yPtJwQRqGSYC72y7j1dpJzt1
Start collection
Add field
createdAt: April 26, 2026 at 11:09:30 PM UTC+3
displayName: "ameer alghamdi"
email: "a0505668500@gmail.com"
firstName: "ameer"
lastName: "alghamdi"
uid: "Dr51yPtJwQRqGSYC72y7j1dpJzt1"
updatedAt: April 26, 2026 at 11:09:30 PM UTC+3
savedPlans > c00P5yiVkJQc...
(default)
Start collection
activities
contactMessages
savedPlans
users
savedPlans
Add document
A1bsegiQSUBN0xXm389w
HToMLr3JWyLncVOm3he2
VoaPRVNsy9uU7ouLvtov
c00P5yiVkJQcUPR7i3HI
c00P5yiVkJQcUPR7i3HI
Start collection
Add field
cardLabel: "Best Overall Match"
category: "sport"
description: "A curated night outing for your friends group built around Jeddah Lanes Bowling Center."
essentials
duration: "Half Day (4 Hours)"
groupSize: "Medium (5 guests)"
mobility: "Moderate movement level"
pacing: "Light active rhythm"
heroAlt: "Jeddah Lanes Bowling Center outing cover image"
heroImage: "../media/plan-images/plan-sport.jpg"
investment
items
0
amount: 18
label: "Obhur Corniche Gathering"
1



HTML validation and Firestore database evidence summary

8.1 Firebase Admin Initialization

```
const admin = require("firebase-admin");
const path = require("path");

if (!admin.apps.length) {
  const serviceAccountPath = path.join(__dirname, "serviceAccountKey.json");

  if (process.env.GOOGLE_APPLICATION_CREDENTIALS) {
    admin.initializeApp();
  } else {
    const serviceAccount = require(serviceAccountPath);
    admin.initializeApp({
      credential: admin.credential.cert(serviceAccount),
    });
  }
```

```
}

const db = admin.firestore();
module.exports = { admin, db };
```

8.2 Saving Contact Messages

```
async function submitContactMessage(req, res) {
  const validationResult = validateContactMessage(req.body);

  if (!validationResult.success) {
    return res.status(400).json({
      success: false,
      error: validationResult.error,
    });
  }

  const documentRef = await db.collection("contactMessages").add({
    ...validationResult.data,
    createdAt: admin.firestore.FieldValue.serverTimestamp(),
  });

  return res.status(201).json({
    success: true,
    id: documentRef.id,
  });
}
```

Backend contact controller: validation and Firestore save operation

8.3 Saving and Loading Plans

```
async function savePlan(req, res) {
  const documentRef = await db.collection("savedPlans").add({
    ...validationResult.data,
    userId: req.user.uid,
    userEmail: req.user.email || "",
    savedAt: admin.firestore.FieldValue.serverTimestamp(),
  });

  return res.status(201).json({
    success: true,
    id: documentRef.id,
    savedPlanId: documentRef.id,
  });
}

async function getPlans(req, res) {
  const snapshot = await db
    .collection("savedPlans")
    .where("userId", "==", req.user.uid)
    .orderBy("savedAt", "desc")
    .get();

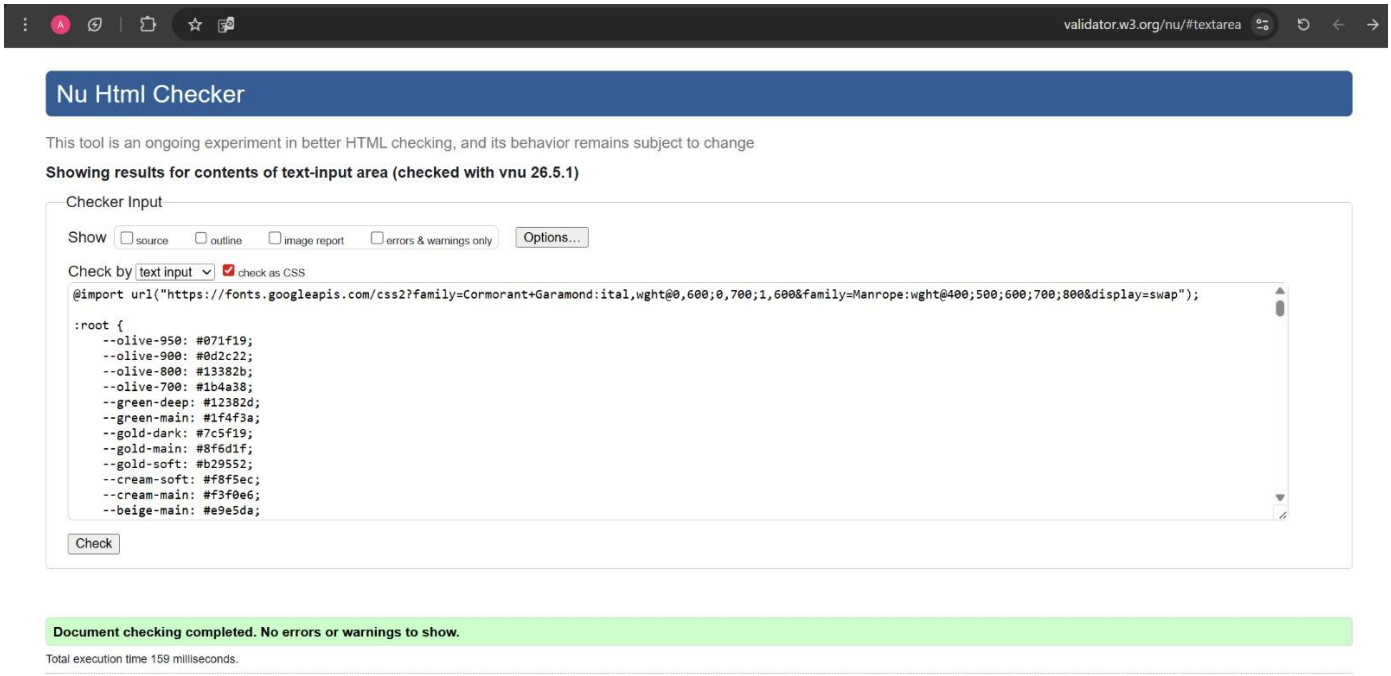
  return res.json({
    success: true,
```

```
    plans: snapshot.docs.map(formatPlanDocument),
  });
}
```

9. HTML Validation Proof

All HTML pages were checked using the W3C Nu HTML Checker. The validation screenshots show that the documents completed checking with no errors or warnings.

- index.html
- about-us.html
- contact-us.html
- main.html
- results.html
- plan-details.html
- saved-plans.html
- authentication.html





Nu Html Checker

This tool is an ongoing experiment in better HTML checking, and its behavior remains subject to change

Showing results for contents of text-input area (checked with vnu 26.4.24)

Checker Input

Show source outline image report errors & warnings only Options…

Check by text input check as CSS

```
                    <li><a href="about-us.html" data-i18n="nav.about">About Us</a></li>
                    <li><a href="contact-us.html" data-i18n="nav.contact">Contact Us</a></li>
                    <li><a href="saved-plans.html" data-i18n="nav.saved">My Plans</a></li>
                </ul>
            </nav>
        </div>
        <p class="footer-bottom" data-i18n="footer.copyright">&#169; 2026 Smart Outing Planner. All rights reserved.</p>
    </footer>

    <script src="../js/i18n.js" defer></script>
    <script src="../js/app.js" defer></script>
    <script src="../js/plan-storage.js" defer></script>
    <script src="../js/saved-plans.js" defer></script>
</body>
</html>
```

Check

Document checking completed. No errors or warnings to show.

Used the HTML parser.

Total execution time 5 milliseconds.



Nu Html Checker

This tool is an ongoing experiment in better HTML checking, and its behavior remains subject to change

Showing results for contents of text-input area (checked with vnu 26.4.20)

Checker Input

Show source outline image report errors & warnings only Options…

Check by text input check as CSS

```
                    <li><a href="contact-us.html" data-i18n="nav.contact">Contact Us</a></li>
                    <li><a href="saved-plans.html" data-i18n="nav.saved">My Plans</a></li>
                </ul>
            </nav>
        </div>

        <p class="footer-bottom" data-i18n="footer.copyright">&#169; 2026 Smart Outing Planner. All rights reserved.</p>
    </footer>

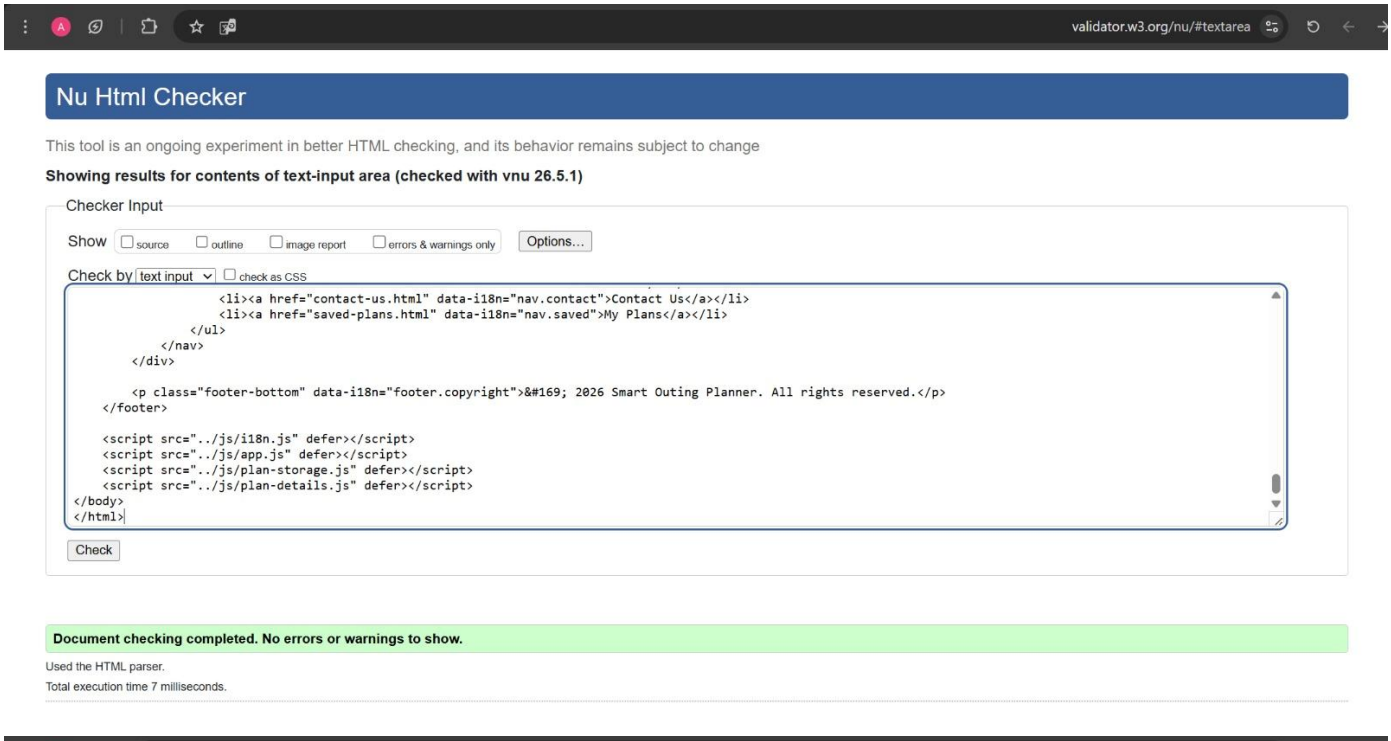
    <script src="../js/i18n.js" defer></script>
    <script src="../js/app.js" defer></script>
    <script src="../js/plan-storage.js" defer></script>
    <script src="../js/planner-results.js" defer></script>
</body>
</html>
```

Check

Document checking completed. No errors or warnings to show.

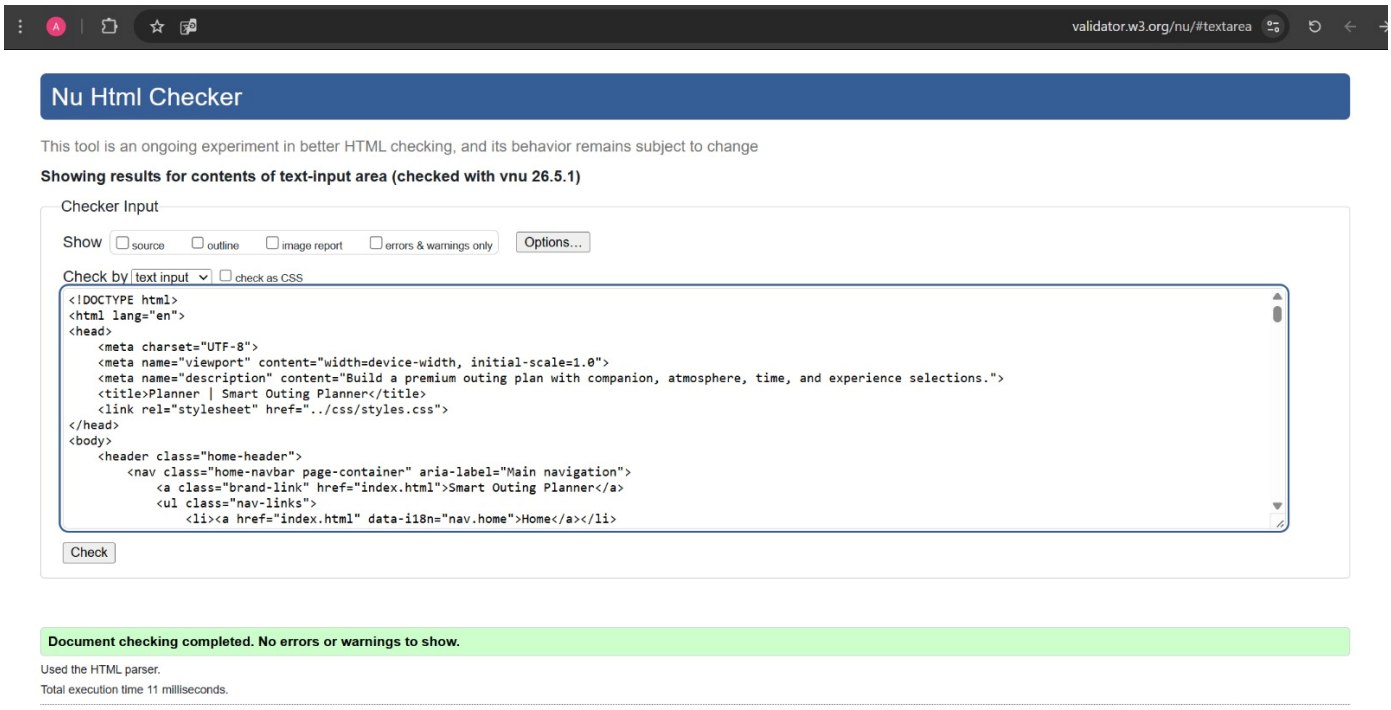
Used the HTML parser.

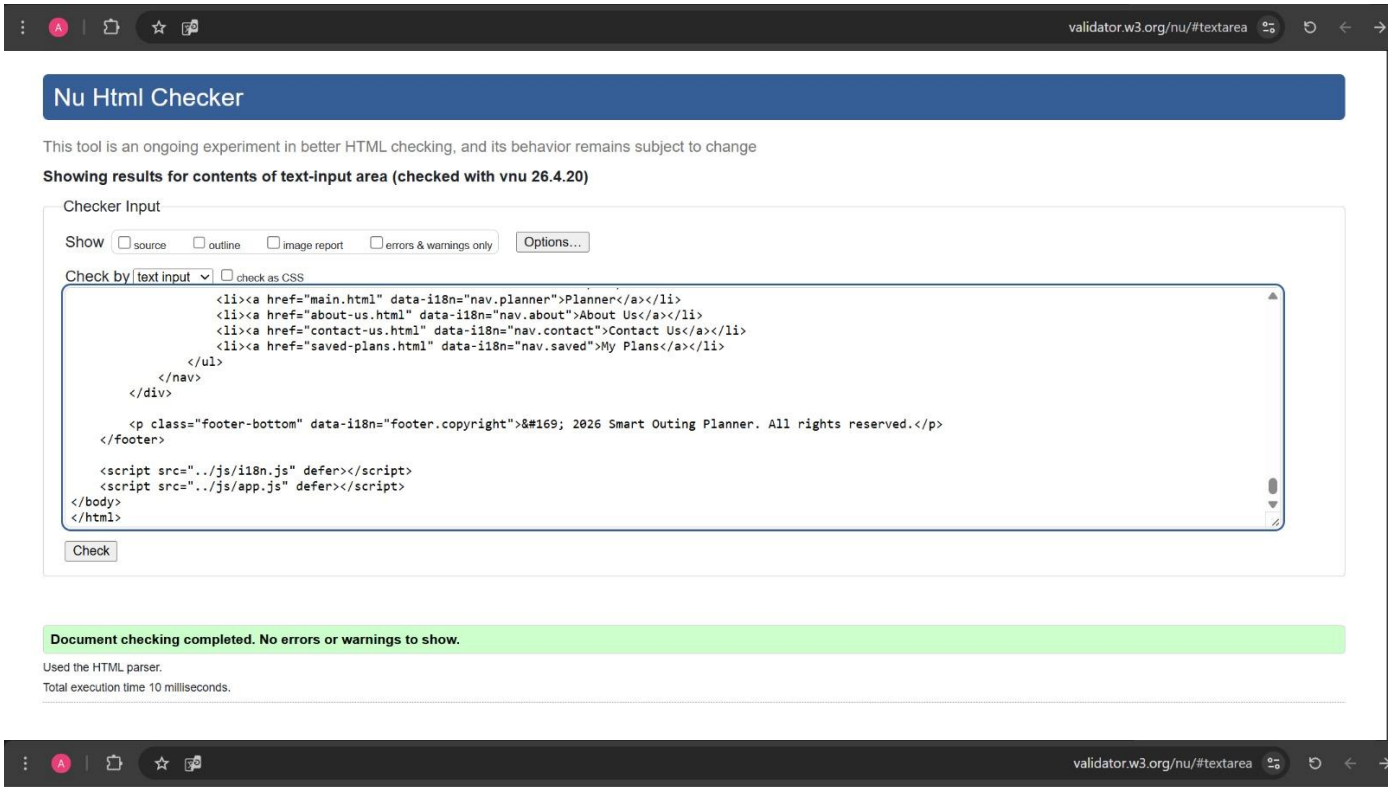
Total execution time 6 milliseconds.


validator.w3.org/nu/#textarea
Nu Html Checker
This tool is an ongoing experiment in better HTML checking, and its behavior remains subject to change
Showing results for contents of text-input area (checked with vnu 26.5.1)
Checker Input
Show source outline image report errors & warnings only Options…
Check by text input check as CSS
Contact Us
My Plans

</nav>

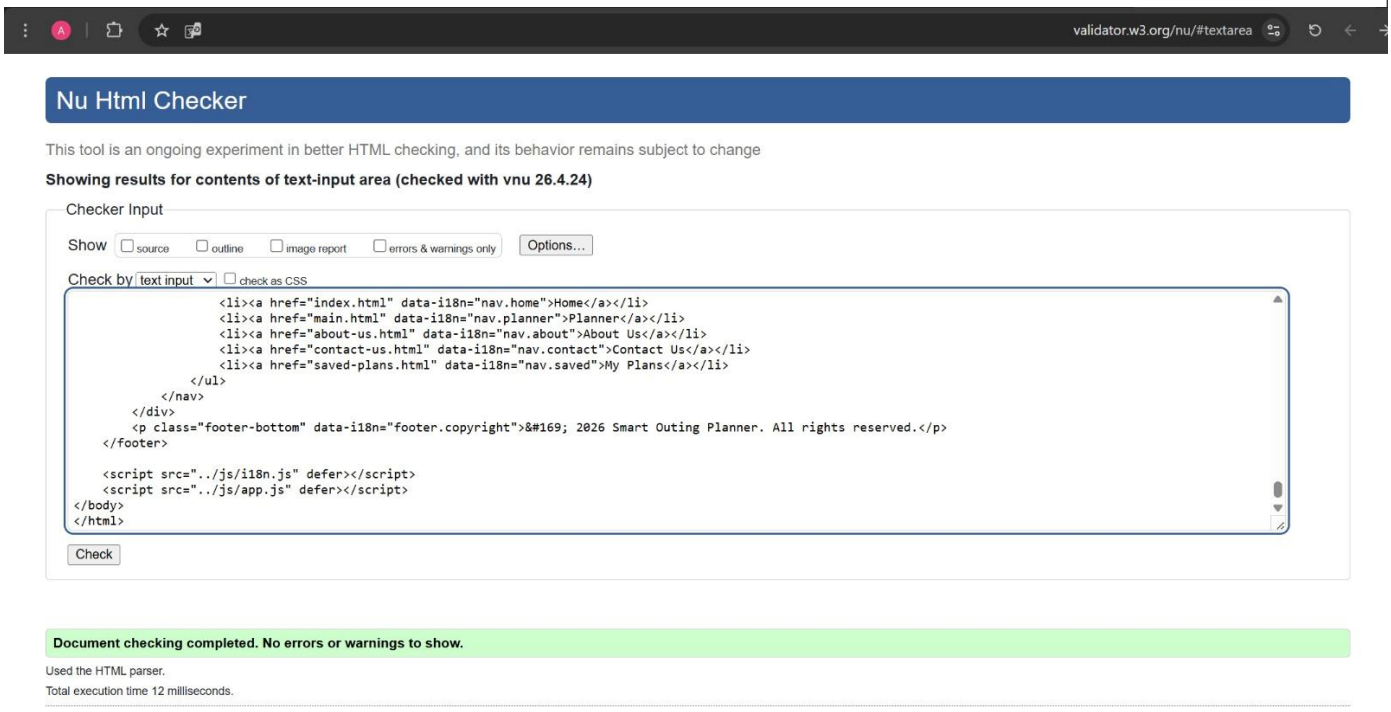
© 2026 Smart Outing Planner. All rights reserved.
</footer>
<script src="../js/i18n.js" defer></script>
<script src="../js/app.js" defer></script>
<script src="../js/plan-storage.js" defer></script>
<script src="../js/plan-details.js" defer></script>
</body>
</html>
Check
Document checking completed. No errors or warnings to show.
Used the HTML parser.
Total execution time 7 milliseconds.


validator.w3.org/nu/#textarea
Nu Html Checker
This tool is an ongoing experiment in better HTML checking, and its behavior remains subject to change
Showing results for contents of text-input area (checked with vnu 26.5.1)
Checker Input
Show source outline image report errors & warnings only Options…
Check by text input check as CSS
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<meta name="description" content="Build a premium outing plan with companion, atmosphere, time, and experience selections.">
<title>Planner | Smart Outing Planner</title>
<link rel="stylesheet" href="../css/styles.css">
</head>
<body>
<header class="home-header">
<nav class="home-navbar page-container" aria-label="Main navigation">
Smart Outing Planner
<ul class="nav-links">
Home
Check
Document checking completed. No errors or warnings to show.
Used the HTML parser.
Total execution time 11 milliseconds.


validator.w3.org/nu/#textarea
Nu Html Checker
This tool is an ongoing experiment in better HTML checking, and its behavior remains subject to change
Showing results for contents of text-input area (checked with vnu 26.4.20)
Checker Input
Show source outline image report errors & warnings only Options…
Check by text input check as CSS
Planner
About Us
Contact Us
My Plans

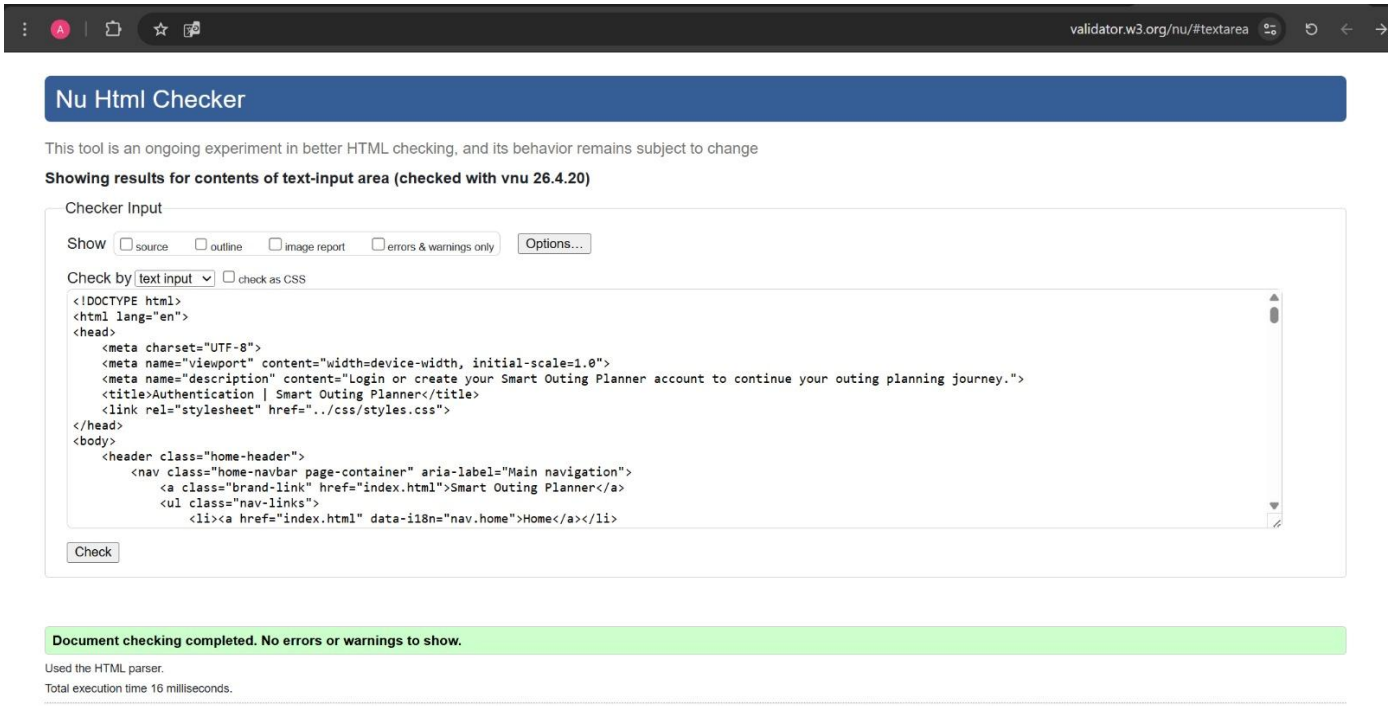
</nav>

© 2026 Smart Outing Planner. All rights reserved.
</footer>
<script src="../js/i18n.js" defer></script>
<script src="../js/app.js" defer></script>
</body>
</html>
Check
Document checking completed. No errors or warnings to show.
Used the HTML parser.
Total execution time 10 milliseconds.


validator.w3.org/nu/#textarea
Nu Html Checker
This tool is an ongoing experiment in better HTML checking, and its behavior remains subject to change
Showing results for contents of text-input area (checked with vnu 26.4.24)
Checker Input
Show source outline image report errors & warnings only Options…
Check by text input check as CSS
Home
Planner
About Us
Contact Us
My Plans

</nav>

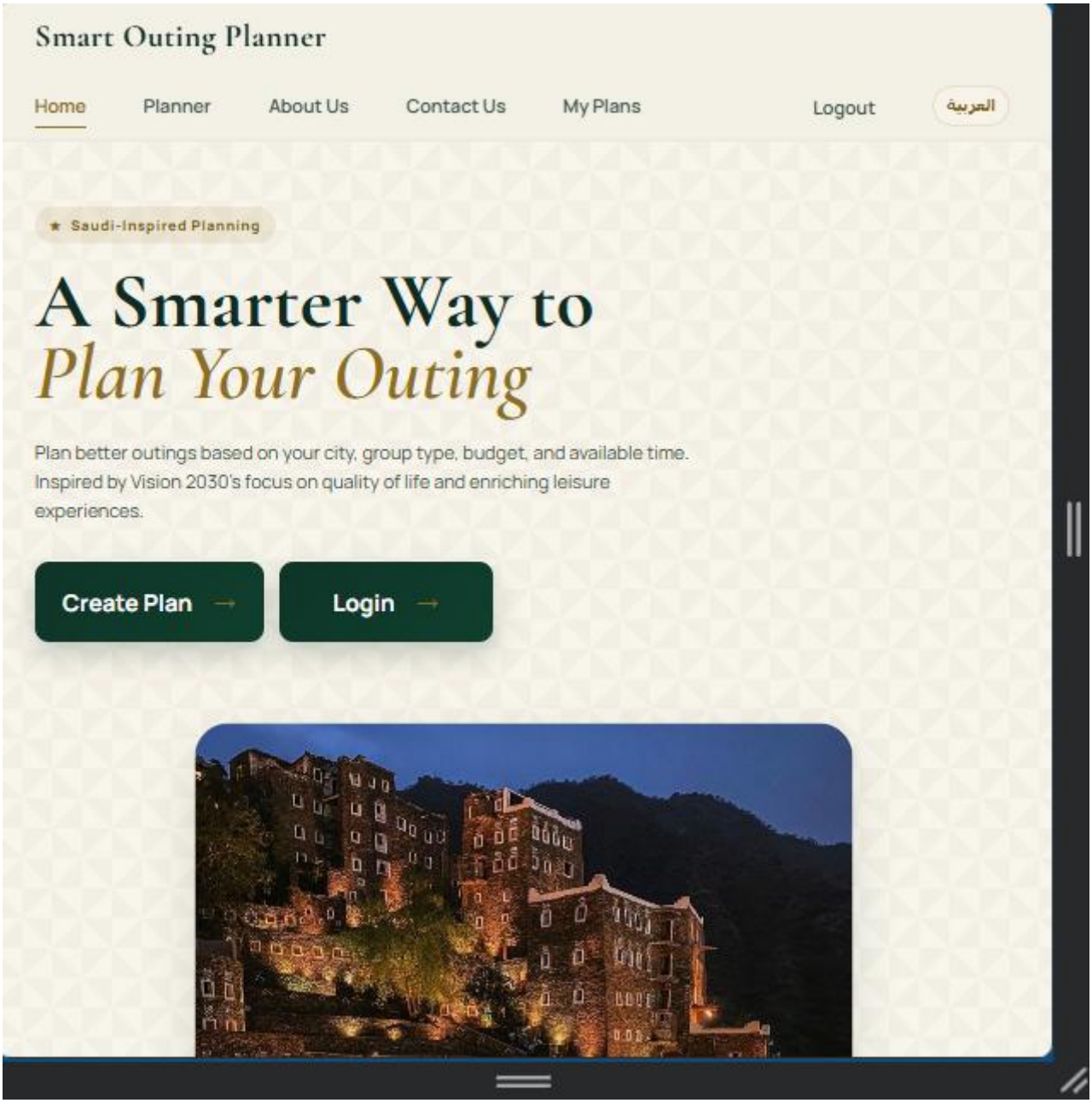
© 2026 Smart Outing Planner. All rights reserved.
</footer>
<script src="../js/i18n.js" defer></script>
<script src="../js/app.js" defer></script>
</body>
</html>
Check
Document checking completed. No errors or warnings to show.
Used the HTML parser.
Total execution time 12 milliseconds.

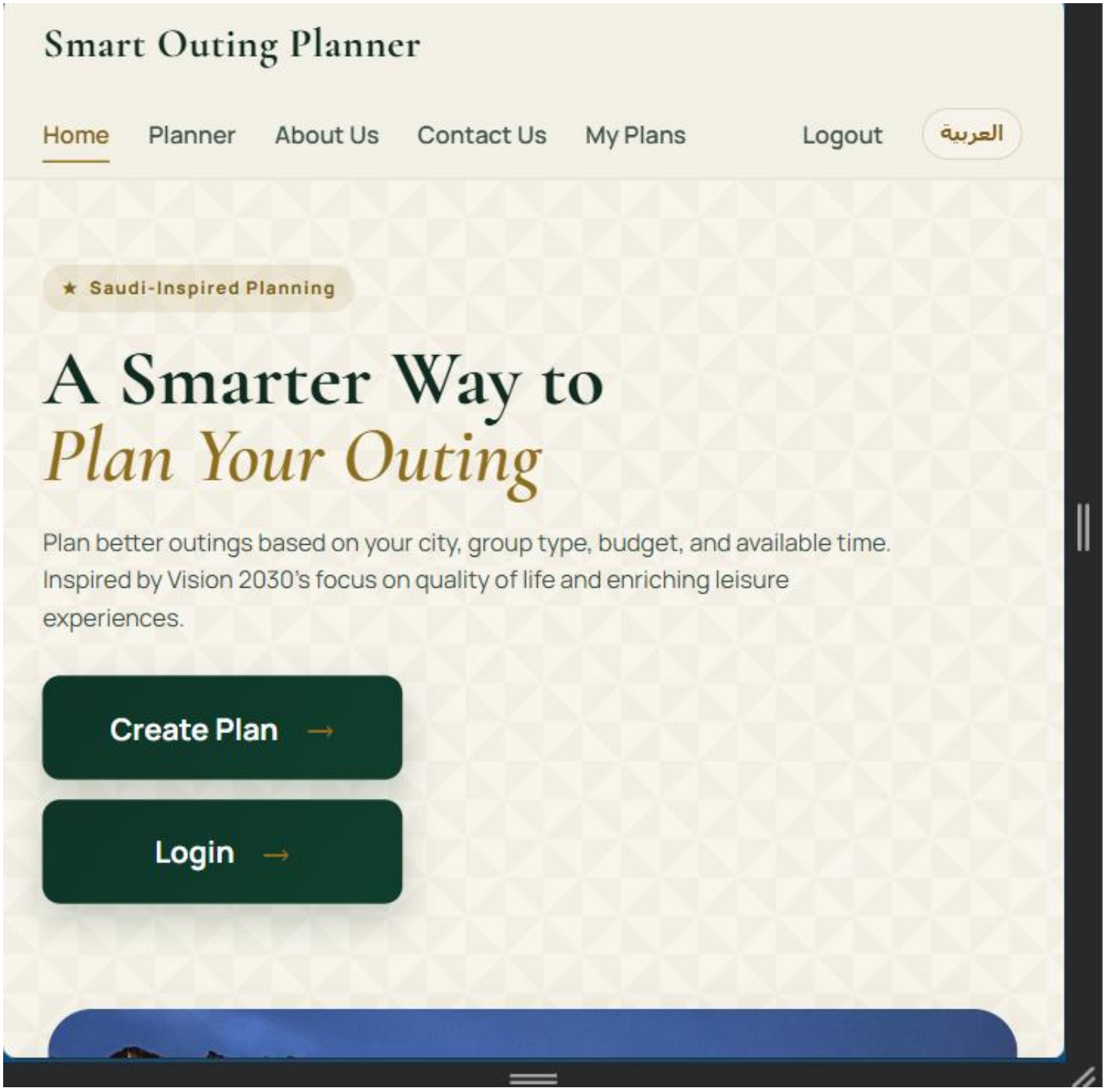


HTML validation screenshots showing no errors or warnings

10. Responsive Design

The website was tested at the required standard screen widths: 576px, 768px, and 992px. The layout adapts by resizing navigation, cards, form sections, images, and content blocks for smaller and larger screens.


Smart Outing Planner
Home
Planner
About Us
Contact Us
My Plans
Logout
العربية
Saudi-Inspired Planning
A Smarter Way to
Plan Your Outing
Plan better outings based on your city, group type, budget, and available time. Inspired by Vision 2030's focus on quality of life and enriching leisure experiences.
Create Plan
Login


Smart Outing Planner
Home
Planner
About Us
Contact Us
My Plans
Logout
العربية
★ Saudi-Inspired Planning
A Smarter Way to
Plan Your Outing
Plan better outings based on your city, group type, budget, and available time. Inspired by Vision 2030's focus on quality of life and enriching leisure experiences.
Create Plan →
Login →



Responsive design screenshot at a small screen width

11. Accessibility and Good Design Practices

- Images include alt text for accessibility.
- Navigation uses semantic nav structure.
- Pages use HTML5 semantic tags such as header, main, section, article, aside, and footer.
- Forms use labels connected to inputs.
- Validation errors are displayed clearly to the user.
- Design uses consistent colors, spacing, typography, and card layouts.
- The interface supports Arabic/English language switching.

12. Testing Summary

- Account creation and login were tested.
- Planner preference selection was tested.
- Generated plans were displayed successfully.
- Replace activity and remove activity features were tested.
- Save plan and saved plans retrieval were tested.
- Plan details page was tested.
- Contact form was tested with invalid and valid inputs.
- Firestore collections were verified.
- HTML validation was checked with the Nu HTML Checker.
- Responsive layouts were tested at the required widths.

13. Conclusion

Smart Outing Planner meets the final project goals by combining a unique website idea, structured multi-page frontend, separated CSS and JavaScript, backend APIs, frontend and backend validation, authentication, database storage, and responsive design.

The project goes beyond a static website by allowing users to generate, refine, save, and revisit outing plans. It demonstrates practical use of HTML5, CSS3, JavaScript, Node.js, Express, Firebase Authentication, and Firestore database integration in one complete working web application.